

Práctica 03: Queries Optimizadas

Objetivo

Aprender a optimizar queries con relaciones usando **eager loading** para evitar el problema N+1.

Escenario

Tienes un blog con Authors, Posts y Tags. Necesitas:

- Listar posts con su autor y tags (sin N+1)
 - Filtrar posts por tag
 - Ordenar por campos de relaciones
-

Instrucciones

Paso 1: Entender el Problema N+1

Abre `starter/main.py` y ejecuta `demonstrate_n_plus_1()` :

```
def demonstrate_n_plus_1():
    """Muestra el problema N+1"""
    db = SessionLocal()

    # 1 query para obtener posts
    posts = db.execute(select(Post)).scalars().all()

    for post in posts:
        # N queries adicionales (1 por cada post)
        print(f"{post.title} by {post.author.name}")

    db.close()
```

Observa cuántas queries se generan (mira los logs SQL).

Paso 2: Solución con `joinedload()`

Descomenta `optimized_with_joinedload()` :

```
from sqlalchemy.orm import joinedload

def optimized_with_joinedload():
    """Usa JOIN para cargar autor en una sola query"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .options(joinedload(Post.author)) # ← Eager loading
    )
    posts = db.execute(stmt).scalars().unique().all()

    for post in posts:
        # Sin queries adicionales!
        print(f"{post.title} by {post.author.name}")
```

```
db.close()
```

Nota: Usa `.unique()` cuando usas `joinedload()` para evitar duplicados.

Paso 3: Cargar Colecciones con `selectinload()`

Para relaciones 1:N (lado muchos) o N:M, usa `selectinload()` :

```
from sqlalchemy.orm import selectinload

def load_with_selectinload():
    """Usa SELECT IN para cargar tags eficientemente"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .options(
            joinedload(Post.author),      # JOIN para 1 objeto
            selectinload(Post.tags)       # SELECT IN para lista
        )
    )
    posts = db.execute(stmt).scalars().unique().all()

    for post in posts:
        tag_names = [t.name for t in post.tags]
        print(f"{post.title} - Tags: {tag_names}")

    db.close()
```

Paso 4: Filtrar por Relación

Para filtrar por campos de una relación, usa JOIN explícito:

```
def filter_by_relation():
    """Filtra posts por tag"""
    db = SessionLocal()

    # Posts con tag "python"
    stmt = (
        select(Post)
        .join(Post.tags)                  # JOIN explícito
        .where(Tag.name == "python")      # Filtro en Tag
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
    )
    posts = db.execute(stmt).scalars().unique().all()

    print(f"\n📝 Posts con tag 'python':")
    for post in posts:
        print(f"    - {post.title}")
```

```
db.close()
```

Paso 5: Ordenar por Relación

```
def order_by_relation():
    """Ordena posts por nombre del autor"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .join(Post.author)          # JOIN necesario para ordenar
        .order_by(Author.name, Post.title) # Ordenar por autor, luego título
        .options(joinedload(Post.author))
    )
    posts = db.execute(stmt).scalars().unique().all()

    print(f"\n📝 Posts ordenados por autor:")
    for post in posts:
        print(f"    - {post.author.name}: {post.title}")

    db.close()
```

Paso 6: Query Compleja - Posts por Autor con Stats

```
from sqlalchemy import func

def author_stats():
    """Estadísticas de posts por autor"""
    db = SessionLocal()

    # Autores con cantidad de posts
    stmt = (
        select(
            Author.name,
            func.count(Post.id).label("post_count")
        )
        .join(Author.posts)
        .group_by(Author.id)
        .order_by(func.count(Post.id).desc())
    )

    results = db.execute(stmt).all()

    print(f"\n📊 Posts por autor:")
    for name, count in results:
        print(f"    - {name}: {count} posts")

    db.close()
```

Paso 7: Comparar Rendimiento

Descomenta `compare_performance()` para ver la diferencia:

```
import time

def compare_performance():
    """Compara queries con y sin optimización"""
    db = SessionLocal()

    # Sin optimización
    start = time.time()
    posts = db.execute(select(Post)).scalars().all()
    for post in posts:
        _ = post.author.name
        _ = [t.name for t in post.tags]
    lazy_time = time.time() - start

    # Con optimización
    start = time.time()
    stmt = select(Post).options(
        joinedload(Post.author),
        selectinload(Post.tags)
    )
    posts = db.execute(stmt).scalars().unique().all()
    for post in posts:
        _ = post.author.name
        _ = [t.name for t in post.tags]
    eager_time = time.time() - start

    print(f"\n🐢 Lazy loading: {lazy_time:.4f}s")
    print(f"🏎️ Eager loading: {eager_time:.4f}s")

    db.close()
```

✅ Checklist de Verificación

- ☐ Entiendo el problema N+1
- ☐ Sé cuándo usar `joinedload()` vs `selectinload()`
- ☐ Puedo filtrar por campos de relaciones
- ☐ Puedo ordenar por campos de relaciones
- ☐ Uso `.unique()` cuando es necesario

🎯 Reto Extra

- Encuentra los 3 tags más usados
- Lista posts que NO tienen tags
- Encuentra autores que no tienen posts

[← Anterior: Práctica 02](#) | [Siguiente: Práctica 04 →](#)